Realcryptik: Pragmatic Election Verification with Zeeperio

Aikam Malhotra[1[]], Aleksander Essex[1[0000−0002−0228−0371]], and Jeremy Clark[2[0000−0002−3533−5965]]

[1] Western University, London, Canada
{amalho26,aessex}@uwo.ca
[2] Concordia University, Montreal, Canada
j.clark@concordia.ca

Abstract. The size of a Zeeperio proof is constant in the number of ballots and voters in an election, as is the time to verify it. It is compact enough for autonomous verification by a smart contract on a blockchain, such as Ethereum.

Keywords: end-to-end verifiable voting, zero-knowledge proofs, zk-SNARKs, polynomial interactive oracle proofs, smart contracts

1 Introduction

Decades of research on end-to-end verifiable (E2E-V) elections have shown strong cryptographic guarantees can be realized for protecting the integrity of elections and the secrecy of voter ballots [15]. In more recent years, pilot projects and deployments of E2E-V platforms have also demonstrated that these systems are usable and scalable to an appropriate degree [2, 7, 4, 11]. Despite these efforts, the vendors of systems for governmental elections continue to run unverifiable elections where the content of individual ballots can be determined by at least one trusted entity in the platform.

In this paper, we contribute to a broader discussion the cryptographic voting community has been having for some number of years: do we compromise our ideals and offer watered down systems that the industry might find more palatable if they offer tangible security advantages over the status quo? We relate this to the term Realpolitik, coined in 19th century Germany to describe statecraft based on pragmatic calculations rather than ideology, and propose *Realcryptik* to describe such a pragmatic approach to election verification.

Flavours of Realcryptik already permeate the research literature. Despite opposition to internet voting, Helios [1] is perhaps the most used E2E-V voting technology. While researchers argue coercion resistance is essential for internet voting, Helios provides none. Scantegrity II [8] was the first E2E-V system to see governmental use. It is built around the compromise that in a paper ballot system, it is ok for optical scanner to learn how the voter voted, side-stepping the complex ballot mechanics its predecessors to avoid this. In recent years, serious

discussions have arose around whether E2E-V should be largely abandoned in favour of risk limiting audits? Should we affirm the use of ballot marking devices when we know most voters do not carefully check their output?

We propose a different compromise in this paper: let EAs and their vendors see the raw ballots, while protecting them from all outsiders (receipt-freeness). For the EA, we still rely on legal protections implemented through procedures but not offer cryptographic protection. Should add more arguments...

The research question of this paper is how to modify the cryptography of an E2E-V voting system to provide integrity, without the need to maintain ballot secrecy from the EA (but still provide receipt-freeness otherwise). The simpler threat model means we can streamline the cryptography, in particular we can prove that encrypted (or committed in our case) ballots are tallied correctly rather than computing the tally under encryption. This shifts the protocol away from multiparty computation and threshold cryptography to essentially an all encompassing zero knowledge proof (ZKP). While ZKPs have been used in voting systems from almost their invention in 1980s and onward, we focus on modern succinct ZKPs called zk-SNARKs.

Zeeperio. We apply this modification to an existing E2E-V back-end called Eperio [12]. Eperio is a simplification of Scantegrity II's backend (known as a switchboard) and is design to be simple enough that a non-expert validator can audit the election using a spreadsheet, such as MS Excel. Eperio's security was studied in detail using formal methods at CCS 2021 paper [3]. By replacing Eperio's cut-and-choose based proof with a zk-SNARK, it obviously is no longer verifiable by hand. However this is a deliberate choice. Traditionally voters would either have to verify an election themselves or rely on a trusted entity (political party, watchdog) to perform this verification—hence the need to simplify elections to a spreadsheet. Today we have systems like Ethereum that provide a peer-to-peer network that is effectively validating that computational outputs of arbitrary code, called smart contracts, are correct. Ethereum protects assets with a market capitalization of $100+B USD and it can protect election results too, as long as the results are verifiable within its very restricted limits (known as gas consumption). If the verification task can be reduced to a succinct (zk-SNARK) proof, voters do not need check the proof at all.

Our contribution is designing a custom succinct argument for Eperio that we call Zeeprio. Our protocol uses polynomial interactive oracle proofs (PIOPs), which have been used to build zk-SNARKs for any arithmetic circuit (and thus any computation), the best known system being Plonk [13]. The size of a Zeeperio proof is constant in the number of ballots and voters in an election, as is the time to verify it. It is compact enough for autonomous verification by a smart contract on a blockchain, such as Ethereum. Zeeperio proofs are linear (as opposed to constant) in the number of candidates, but this seems acceptable as reading the final tally or checking an audited ballot is linear in the number of candidates.

2 Background and Related Work

2.1 Eperio

We assume the reader is already familiar with end-to-end verifiable (E2E-V) voting protocols which allow for: (i) an election authority to prove an election tally is computed correctly without learning individual voter choices, and for (ii) voters to check their individual ballots are included unmodified in the tally without being able to reveal how they voted to anyone else. Eperio is an E2E-V tallying protocol (or 'backend') that composes with several styles of paper ballots, including Pret a Voter and Scantegrity I. A brief description of Eperio is provided in Figure **??** with a Pret a Voter style ballot.

In addition to succinctness, we also clean up a few issues that were not fully resolved in Scantegrity II and Eperio:

- As pointed about by Basin et al, Eperio does not prove ballots were not overvoted [3]; Zeeperio does (§**??**).
- As acknowledged by Essex et al, Eperio does not support Scantegrity II ballots [12]; Zeeperio does (§**??**).
- As acknowledged by Chaum et al, Scantegrity II supports dispute resolution but it requires all confirmation codes on the ballot to be revealed [8, 9]. Zeeprio offers a simple proof of incorrect code without revealing information about all codes (§**??**).

A remaining issue, pointed out by Basin et al [3], are clash attacks [16] which are not explicitly addressed by Eperio. However they are applicable to all major E2E-V voting systems and are addressed through strict polling place policies that denote which ballot is given to which voter.

2.2 Succinct Proofs

While zero knowledge proofs/arguments (ZKPs) date back to the 1980s [14, 5], classic ZKPs produce proofs that are at least linear in the size of the statement being proven. By contrast, succinct zero knowledge arguments (SNARKs) are sublinear, typically with a logarithmic or constant proof size. A Polynomial Interactive Oracle Proof (PIOPs) is a pattern that a number of SNARKs have in common [10, 6]. In this pattern, the prover specifies polynomials and verifiers challenge the prover to open these polynomials at random points. PIOP does not say *how* to make a SNARK, it is just a general pattern.

To realize a PIOP, the prover must send a succinct representation of each polynomial from the prover to the verifier, which commits the prover to those polynomials. When opening a polynomial, the verifier must be certain it is the opening of the same polynomial that was committed to. PIOPs rely on a cryptographic primitive called a polynomial commitment scheme (PCS) to handle this. The rest of a PIOP protocol does not use any cryptography. A number of PCSs have been proposed, the most prominent are KZG (very short, based on

bilinear pairings, requires a trusted setup) and (DEEP-)FRI (longer, based on hash functions, conjectured to be post-quantum secure, no trusted setup). In Zeeperio, we use KZG because it integrates well with Ethereum.

KZG uses a universal trusted setup: a secret value τ is chosen at random, used to compute a structured reference string (SRS) $\left\langle g^{\tau}, g^{\tau^2}, g^{\tau^3}, \ldots \right\rangle$ in a pairing-friendly group defined by generator g, and then erased. Knowledge of τ after the setup is sufficient to fake KZG openings and thus forge false ZKP transcripts. However the setup can be run as an open participation multiparty computation where participants prove their contribution is correct and only one honest participant must forget τ for the SRS to be secure. Ethereum ran an open trusted setup in 2023, accepting 100K contributions over 208 days. A single SRS, such as this, can be shared by everyone wanting to do a KZG commitment (known a universal trusted setup).

In practice, zk-SNARKs for a specific computation can be deployed in different ways: (a) written in a high-level language and compiled to VM instructions, (b) written in special markup languages and compiled into arithmetic circuits, or (c) written as a custom arithmetic circuit. In each case, these representations are fed into a general purpose proof system for circuits or zkVMs. Each level of abstraction adds overhead and costs to the prover. Our approach is 'closer to the metal.' We do not use circuits or Plonk itself. We instead go underneath what Plonk provides and use the same kind of building blocks it uses to make a custom PIOP argument for Zeeperio.

2.3 Smart Contracts

Since smart contracts can implement arbitrary code, they can support cryptographic operations however this approach is usually prohibitively expensive. Ethereum offers cheaper 'pre-compiles' for pairing-based operations on the elliptic curve `alt-bn128`. On Ethereum's testnet, support for curve `bls12-381` has been staged for future deployment, as well as KZG-specific operations over `bls12-381`, using the fore-mentioned SRS. Zeeperio on `alt-bn128` or `bls12-381`

be added as well KZG opening proofs.

in the future,

BN128

2.4 Zeeperio: A succinct, On-Chain Approach

Zeeperio shifts from statistical audits to cryptographic arguments of knowledge. At a high level, Zeeperio uses the Poly-IOP model to encodes the entire state of an election as a small set of polynomials. Instead of writing a generic arithmetic circuit, Zeeperio works directly at the polynomial level to design custom constraints for ballot structure, sums over ballots, and tallies. Then, using the KZG commitment scheme, Zeeperio allows the Election Authority (EA) to generate a single, succinct proof that the election outcome is correct. This proof is constant in size and verification time, regardless of how many voters cast their ballots.

This efficiency allows for the verification logic to be encoded into an Ethereum smart contract. Instead of requiring observers to download data and run software, the *public audit* becomes a transaction on a the blockchain. If the smart contract accepts the proof, the election tally is mathematically guaranteed to be consistent with the committed ballots. This lowers the barrier to verification from running complex cryptographic software to checking a simple status flag on the Ethereum block explorer.

3 Threat Model and Contributions

Before detailing the protocol mechanics, we formalize the operational environment and the specific contributions of this work relative to the state of the art.

3.1 The RealCryptik Threat Model

Zeeprio adopts the *RealCryptik* threat model. Traditional E2E literature often models the EA as a powerful adversary that must be prevented from learning individual votes at all costs. However, this model often ignores the reality of elections. In many scenarios, the EA has the full capability to deanonymize votes.

The RealCryptik model explicitly assumes that the EA **has access to who voted for whom**. Under this assumption, the primary security goal shifts from absolute privacy to absolute **integrity**. The system must guarantee that even an EA with full view of voter choices cannot manipulate an election without detection. This model reflects the reality of *black box* vendors while enforcing the verification constraints necessary for public trust. While public privacy is maintained through zero-knowledge proofs, the protocol also prevents the EA from manipulating the outcome based on their knowledge.

3.2 Contributions

This work presents the following contributions.

Succinct universal verification: All election constraints are encoded as a single Poly-IOP relation and proven via a zkSNARK proof, independent of the number of voters.

Enhanced privacy and dispute resolution: Confirmation codes are committed to a single polynomial and are only selectively opened when needed. After a voter has cast their ballot, they receives a receipt containing a confirmation code to which they can check it appears in the public bulletin board. This is done in a way that prevents the receipt from being used as a proof of how the voter voted.

On-chain verification: The final proof is verified by an Ethereum smart contract. This removes the need for a proprietary verifier software and anchors the election result in a public, tamper-evident ledger.

4 Preliminaries

We now introduce the cryptographic tools used in Zeeperio.

4.1 Polynomial interactive oracle proofs

A polynomial interactive oracle proof (Poly-IOP) is an interactive proof where the prover sends commitments to polynomials and the verifier has oracle access to these polynomials through evaluation queries. Formally, a Poly-IOP protocol for a relation $\mathcal{R}$ between public input x and witness w follows:

(i) The prover, given x and w, computes polynomials $f_1, \ldots, f_m$ and sends commitments to them.
(ii) The verifier sends random challenges $r_1, \ldots, r_\ell$.
(iii) The prover answers with evaluations $f(r_\ell)$ and proofs of correct evaluation.
(iv) The verifier checks the evaluations satisfy a system of polynomial identities and the evaluation proofs are valid.

If all checks pass, the verifier accepts the statement *there exists w such that* $(x, w) \in \mathcal{R}$.

4.2 KZG polynomial commitments

Zeeperio uses the KZG (Kate-Zaverucha-Goldberg) polynomial commitment scheme. We recall its interface at a high level. A trusted setup is used to produces a structured reference string (SRS) consisting of group elements

$$\{g, g^\tau, g^{\tau^2}, g^{\tau^d}\} \in G_1, \qquad \{h, h^\tau\} \in G_2,$$

for a secret τ.

Given a polynomial $f(X) = \sum_{i=0}^{d} f_i X^i$, the commitment is

$$\mathsf{Com}(f) = \sum_{i=0}^{d} f_i [\tau^i] \in G_1.$$

To open $\mathsf{Com}(f)$ at a point $z \in \mathbb{F}$, the prover computes $y = f(z)$ and the quotient

$$q(X) = \frac{f(X) - y}{X - z}$$

and returns an opening proof $\pi = \mathsf{Com}(q)$. The verifier checks that

$$e(\mathsf{Com}(f) - y, h) = e(\pi, [h^\tau - z]),$$

where $e : G_1 \times G_2 \to G_T$ is a bilinear pairing.

In Zeeperio we use KZG both to commit to the election polynomials. We combine multiple constraints into a single quotient polynomial using random coefficients and prove that it vanishes on Ω by checking a single evaluation.

4.3 Zero-knowledge and Fiat–Shamir

We can apply the Fiat-Shamir heuristic to the Poly-IOP to make the protcol non-interactive. This can be done by replacing the verifier's random challenges by outputs of a hash function on the transcript. To achieve zero-knowledge, we add random blinding polynomials to all witness polynomials before committing, and we ensure that any information opened to the verifier is public. Therefore, the only meaningful information revealed by the proof is that the constraints are satisfied, while keeping the underlying ballot structure and confirmation codes hidden.

4.4 Ethereum verification model

Finally, we briefly outline the on-chain verification model. Ethereum provides precompiled contracts for the BN254 pairing and scalar multiplication. The verification contract uses the SRS and the structure of the combined pairing equation corresponding to the KZG openings. Given the public input and the proof, the contract performs a simple pairing check. If the check passes, the election result has been cryptographically verified.

5 The Zeeperio Protocol

Election data and domain. We consider an election with $\mathcal{B}$ ballots and $\mathcal{C}$ candidates, with $\mathcal{P} = \mathcal{B} \cdot \mathcal{C}$ ballot-candidate positions listed in canonical order. Let $i = j \cdot \mathcal{C} + k$ for ballot index $j \in \{0, \ldots, \mathcal{B} - 1\}$ and candidate index $k \in \{0, \ldots, \mathcal{C} - 1\}$. The EA forms the following columns, each of length $\mathcal{P}$.

1. **Ballot Column ($\mathsf{B_{ID}}$):** Identifies the unique ballot $\mathcal{B}_j$.
2. **Confirmation Code Column ($\mathsf{C_{confirm}}$):** A unique confirmation code issued to each position. The voter receives the code corresponding to their selection.
3. **Mark Column ($\mathsf{M_{ark}}$):** Indicates if the position was selected for vote.
4. **Audit Column ($\mathsf{A_{udit}}$):** Indicates if the ballot $\mathcal{B}_j$ was selected for audit.

For efficient FFT operations, we require an evaluation domain size n that is a power of two. We set $n = 2^t$ where $t = \lceil \log_2 \mathcal{P} \rceil$. The EA pads the columns with zeros for indices i where $\mathcal{P} \leq i < n$. The columns $\mathsf{M_{ark}}$ and $\mathsf{C_{confirm}}$ are witness data and are not published.

Let $\mathbb{F}$ be the prime field used by the commitment scheme and let $\Omega = \{\omega^0, \ldots, \omega^{n-1}\}$ be an evaluation domain with primitive nth root ω. Each column is interpolated into a polynomial $F(X) \in \mathbb{F}[X]_{<n}$ satisfying $F(\omega^i) = F[i]$ on Ω.

Commitments and Fiat-Shamir. Zeeperio bases itself as a Poly-IOP using the KZG polynomial commitments with $\mathsf{KZG.Commit}(F)$, $\mathsf{KZG.Open}(F, z)$, and $\mathsf{KZG.Verify}(C, z, y, \pi)$, and derives all verifier challenges via Fiat-Shamir.

Zero-knowledge. We use the hiding variant of KZG commitments (commit as $g^{F(\tau)}h^r$), and off-domain masking (randomize $F'(X) = F(X) + \rho \cdot Z_H(X)$ where $Z_H(X) = X^n - 1$ vanishes on Ω), so commitments and openings do not leak vote or receipt information.

5.1 Setup

In the setup phase, the EA performs the following steps.

(S1) Obtains KZG SRS from a publicly run Ethereum Powers-of-Tau ceremony $\mathsf{SRS} = (g, g^{\tau}, g^{\tau^2}, \ldots, g^{\tau^d})$, supporting degree $\leq d$ commitments.
(S2) Fixes the finite field $\mathbb{F}_q$, root-of-unity domain Ω of size n, and encoding of election data into polynomials.
(S3) Generates a set of unique confirmation codes and assigns each $\mathsf{C}_c onfirm$ code to each position on the ballot.
(S4) Deploys an Ethereum smart contract that uses the SRS and the verification logic for the Zeeperio proof.

The protocol uses a universal SRS from a Powers-of-Tau ceremony,

5.2 Main Election Proof

Goal. The EA proves in zero-knowledge that the committed election columns satisfy:

- well-formed audit marks
- well-formed cast marks
- audit/mark exclusion
- published tally is consistent with the mark column

Constraint family as vanishing checks. We define a family of low-degree polynomials $\{P_i(X)\}$ such that the election is valid iff

$$P_i(\omega^t) = 0 \quad \forall \omega^t \in \Omega \text{ and for all } i.$$

These constraints encode the following, each as a polynomial identity that vanishes on Ω:
Audit column $\mathsf{A}_{\mathsf{udit}}$:

1. The padded values must be set to $\mathsf{A}_{\mathsf{udit}}[i] = 0$ for $i \geq P$.
2. All values should either be set to 1, or 0 $\mathsf{A}_{\mathsf{udit}}[i] \in \{0, 1\}$.
3. For each ballot j, the $\mathcal{C}$ entries $\mathsf{A}_{\mathsf{udit}}[j\mathcal{C}], \ldots, \mathsf{A}_{\mathsf{udit}}[j\mathcal{C} + \mathcal{C} - 1]$ are equal. Additionally, the block-sum for each $\mathsf{A}_{\mathsf{udit}}$ block is $\in \{0, C\}$.
4. The number of 1s in $\mathsf{A}_{\mathsf{udit}}$ is equal to the published number of audited ballots $\mathsf{Sum}_{\mathsf{A}} \cdot \mathcal{C}$ using an accumulator constraint.

Mark column $\mathsf{M}_{\mathsf{ark}}$:

1. The padded values must be set to $\mathsf{M_{ark}}[i] = 0$ for $i \geq P$.
2. All values should either be set to 1, or 0 $\mathsf{M_{ark}}[i] \in \{0, 1\}$.
3. The block-sum for each $\mathsf{M_{ark}}$ block is $\in \{0, C\}$.
4. The total number of marks is equal to the published number of ballots cast $\mathsf{Sum_M}$, enforcing one mark per cast ballot using an accumulator.

Audit and Mark Exclusion: No ballot must be both audited and cast $\mathsf{A_{udit}}[i] \cdot \mathsf{M_{ark}}[i] = 0$ for all $i < n$.
Tally correctness: For each candidate $c \in \{0, \ldots, \mathcal{C} - 1\}$, the published tally Sum_c is equal to the sum of $\mathsf{M_{ark}}[i]$ over the stream $\{c, c+\mathcal{C}, c+2\mathcal{C}, \ldots\}$, enforced by a strided accumulator (shift by $\omega^{\mathcal{C}}$) with boundary conditions for the first/last stream elements.

Batched quotient argument. Let $k = 17 + \mathcal{C}$ be the number of vanishing polynomials. Using Fiat-Shamir, derive $\alpha \xleftarrow{\$} \mathbb{F}$ and define the batched vanishing polynomial

$$P_{\mathsf{vanish}}(X) = \sum_{i=1}^{k} \alpha^{i-1} P_i(X).$$

Then form the quotient

$$Q(X) = \frac{P_{\mathsf{vanish}}(X)}{Z_H(X)} \quad \text{where } Z_H(X) = X^n - 1.$$

If all constraints hold on Ω, $Q(X)$ exists as a polynomial.

Proof. The EA commits to the masked witness and auxiliary polynomials required by the constraints ($\mathsf{A_{udit}}, \mathsf{M_{ark}}$ and the block-sum/accumulator polynomials) and to Q, then opens them at a single Fiat-Shamir point $\zeta \in \mathbb{F}$.

Verification. From the opened values at ζ, the verifier reconstructs $P_{\mathsf{vanish}}(\zeta)$ and checks

$$P_{\mathsf{van}}(\zeta) \stackrel{?}{=} Q(\zeta) \cdot (\zeta^n - 1),$$

accepting iff the check passes under $\mathsf{KZG.Verify}$.

5.3 Voter Verification Proofs

Voter verification is a *separate* zero-knowledge proof from the main election proof, but it reuses the same setup and the same committed election polynomials.

Print audit (spoiled ballots). Given a spoiled ballot $\mathcal{B}_j$, the EA opens the commitments for the corresponding index block $\{j\mathcal{C}, \ldots, j\mathcal{C} + \mathcal{C} - 1\}$ to show the ballot was audited and not cast. There is no need to satisfy the receipt-freeness requirement for audited ballots.

Ballot inclusion (receipt check without revealing the choice). To show a voter's receipt code corresponds to a marked position without revealing which candidate was marked, the EA proves a permutation relation over tuples. For each i, form $(b_i, c_i, m_i) = (\mathsf{B_{ID}}[i], \mathsf{C_{confirm}}[i], \mathsf{M_{ark}}[i])$ and a shuffled copy $(b'_i, c'_i, m'_i) = (b_{\sigma(i)}, c_{\sigma(i)}, m_{\sigma(i)})$ for a secret shuffle σ. Using Fiat-Shamir, sample $\alpha, r \xleftarrow{\$} \mathbb{F}$, encode tuples into scalars $v_i = b_i + \alpha c_i + \alpha^2 m_i$ and $v'_i = b'_i + \alpha c'_i + \alpha^2 m'_i$, define $V_1[i] = r - v_i$ and $V_2[i] = r - v'_i$, and enforce multiset equality via a grand product accumulator argument. All resulting checks are expressed as a small set of vanishing polynomials, batched with a random β, and verified with a single quotient test exactly as in the main proof.

After the permutation proof, the EA reveals the *mathcalC* tuples in the shuffled list whose ballot ID equals $\mathcal{B}_j$ via $\mathsf{KZG.Open}$. Exactly one tuple has $m' = 1$, establishing inclusion while the shuffle hides which candidate position was marked.

Receipt correctness (prove a disputed code is not *on ballot $\mathcal{B}_j$).* Let the disputed code be C. The EA defines a helper indicator polynomial $D(X)$ whose evaluations satisfy $D(\omega^i) = 1$ iff $\mathsf{C_{confirm}}[i] = C$ using *Fermat's Little Theorem.* Let $\mathsf{Sel}_j(X)$ be the public selector that is 1 on indices $\{j\mathcal{C}, \ldots, j\mathcal{C} + \mathcal{C} - 1\}$ and 0 elsewhere. The EA proves

$$z \;=\; \sum_{i=0}^{n-1} \mathsf{Sel}_j(\omega^i) \cdot D(\omega^i) \;=\; 0.$$

This implies no confirmation code on ballot $\mathcal{B}_j$ is equal to the disputed code C without revealing any of the ballot's codes.

6 Security Analysis

6.1 Completeness

Theorem 1 (Completeness). *If an honest Prover $\mathcal{P}$ computes the accumulators $Acc_{T_1}(X)$ and $Acc_{T_2}(X)$ according to the backward accumulation rule defined in the protocol, then the Verifier $\mathcal{V}$ accepts with probability 1.*

Proof. The honest prover constructs the backward accumulator such that $Acc_{T_1}(\omega^i) = \prod_{j=i}^{n-1} T_1(\omega^j)$. The protocol enforces constraints via a batched vanishing polynomial $P_{vanish}(X)$.

1. **Boundary Consistency:** At the last element $X = \omega^{n-1}$, the accumulator is defined as $Acc_{T_1}(\omega^{n-1}) = T_1(\omega^{n-1})$. Consequently, the term $(Acc_{T_1}(X) - T_1(X))$ is zero at this point, satisfying the boundary constraint $P_{vanish1}$.
2. **Recursive Consistency:** For all $X \in \Omega$ except $X = \omega^{n-1}$, the recursive definition holds: $Acc_{T_1}(\omega^i) = T_1(\omega^i) \cdot Acc_{T_1}(\omega^{i+1})$. This ensures the constraint $P_{vanish3}$ evaluates to zero on the relevant domain.

3. **Product Equality:** Since the tuples are permutations, the total products over Ω are equal. Thus, $Acc_{T_1}(1) = \prod_{x\in\Omega} T_1(x) = \prod_{x\in\Omega} T_2(x) = Acc_{T_2}(1)$, satisfying the equality constraint $P_{vanish5}$.

Since all individual constraint polynomials vanish on Ω, $P_{vanish}(X)$ is divisible by the vanishing polynomial $Z_H(X) = X^n - 1$. Therefore, the quotient check $P_{vanish}(\zeta) - Q(\zeta)Z_H(\zeta) \stackrel{?}{=} 0$ holds for any $\zeta \in \mathbb{F}_q$. □

6.2 Knowledge Soundness

Theorem 2 (Knowledge Soundness). *The protocol satisfies knowledge soundness in the Algebraic Group Model. If a malicious prover $\mathcal{P}$ convinces $\mathcal{V}$ with non-negligible probability ϵ, there exists an efficient Extractor $\mathcal{E}$ that outputs polynomials $T(X)$ and $T'(X)$ such that the set of their evaluations on Ω are permutations of each other.*

Proof. In the AGM, the Extractor $\mathcal{E}$ observes the commitments sent by $\mathcal{P}$. Since $\mathcal{P}$ outputs valid commitments $[Acc_{T_1}], [Acc_{T_2}]$, it must know the underlying polynomials.

If $\mathcal{V}$ accepts, the equation $P_{vanish}(\zeta) = 0$ holds at a random challenge $\zeta \leftarrow \mathbb{F}_q$. By the Schwartz-Zippel Lemma, this implies that $P_{vanish}(X)$ is divisible by $Z_H(X)$ with overwhelming probability. We unroll these constraints to prove the validity of the statement:

1. **Unrolling the Accumulator:** From the boundary constraint ($P_{vanish1} \equiv 0$), we have $Acc_{T_1}(\omega^{n-1}) = T_1(\omega^{n-1})$. From the constraint ($P_{vanish3} \equiv 0$), for any $x \in \Omega$ except $X = \omega^{n-1}$, $Acc_{T_1}(x) = T_1(x) \cdot Acc_{T_1}(\omega x)$. With this backwards accumulation, we determine that $Acc_{T_1}(1)$ stores the complete product:
$$Acc_{T_1}(1) = \prod_{i=0}^{n-1} T_1(\omega^i) \tag{1}$$
Similarly, we derive $Acc_{T_2}(1) = \prod_{i=0}^{n-1} T_2(\omega^i)$.
2. **Equality of Multisets:** The final constraint ($P_{vanish5} \equiv 0$) ensures $Acc_{T_1}(1) = Acc_{T_2}(1)$. Substituting the definitions of T_1 and T_2:
$$\prod_{x\in\Omega}(r - T(x)) = \prod_{x\in\Omega}(r - T'(x)) \tag{2}$$
This equation implies equality between two polynomials in the variable r. Since r is chosen uniformly at random by the verifier *after* the commitments to $T(X)$ and $T'(X)$, the equality holds identically with overwhelming probability. Therefore, these polynomials are considered to be *set equal*. Thus, T' is a valid permutation of T.

□

6.3 Honest-Verifier Zero-Knowledge

Theorem 3 (HVZK). *The protocol is Honest-Verifier Zero-Knowledge. There exists a Simulator $\mathcal{S}$ that outputs a transcript indistinguishable from a real prover's trace.*

Proof. The Simulator $\mathcal{S}$ operates as follows:

1. **Challenge Sampling:** $\mathcal{S}$ samples random challenges $r, \beta, \zeta \leftarrow \mathbb{F}_q$ exactly as the honest verifier would.
2. **Simulate Commitments:** $\mathcal{S}$ outputs random group elements for $[Acc_{T_1}], [Acc_{T_2}], [T_1], [T_2]$. Due to the hiding property of KZG (with blinding), these are indistinguishable from real commitments.
3. **Simulate Evaluations:**
   - $\mathcal{S}$ chooses random values for evaluations $\bar{T}_1, \bar{T}_2, \overline{Acc}_{T_1}, \overline{Acc}_{T_2}$ at ζ and required shifted points (e.g., $\omega\zeta$). This is valid because the blinded polynomials act as One-Time Pads at any point $\zeta \notin \Omega$.
   - $\mathcal{S}$ must ensure the verification equation holds: $P_{vanish}(\zeta) - Q(\zeta)Z_H(\zeta) = 0$.
   - $\mathcal{S}$ computes the value of $P_{vanish}(\zeta)$ using the randomly sampled evaluations and the formulas for Z_1, Z_2, Z_3 evaluated at ζ.
   - $\mathcal{S}$ then *sets* the evaluation of the quotient polynomial $\bar{Q} = Q(\zeta)$ specifically to satisfy the equation:

     $$\bar{Q} = \frac{P_{vanish}(\zeta)}{Z_H(\zeta)}$$

     Since ζ is random, $Z_H(\zeta) \neq 0$, making this division valid.
4. **Transcript Consistency:** The tuple of commitments, challenges, and evaluations $(\ldots, \bar{Q})$ satisfies the algebraic verification check by construction. Because the evaluations of blinded polynomials are uniformly distributed, the simulated transcript is statistically indistinguishable from a real transcript.

7 Implementation

8 Future Work

We conclude with several directions for extending and deploying Zeeperio.

Integration with optical scanners

Online voting

Acknowledgements. We thank the reviewers who helped to improve our paper. J. Clark acknowledges support for this research project from (i) the National Sciences and Engineering Research Council (NSERC), Raymond Chabot Grant Thornton, and Catallaxy Industrial Research Chair in Blockchain Technologies, (ii) the AMF (Autorité des Marchés Financiers), and (iii) NSERC through a Discovery Grant.

References

1. Adida, B.: Helios: web-based open-audit voting. In: USENIX Security Symposium. pp. 335–348 (2008)
2. Adida, B., Marneffe, O.d., Pereira, O., Quisquater, J.J.: Electing a university president using open-audit voting: Analysis of real-world use of Helios. In: EVT/WOTE (2009)
3. Basin, D., Dreier, J., Giampietro, S., Radomirović, S.: Verifying table-based elections. In: ACM CCS (2021)
4. Bell, S., Benaloh, J., Byrne, M.D., DeBeauvoir, D., Eakin, B., Kortum, P., McBurnett, N., Pereira, O., Stark, P.B., Wallach, D.S., et al.: {STAR-Vote}: A secure, transparent, auditable, and reliable voting system. In: 2013 Electronic Voting Technology Workshop/Workshop on Trustworthy Elections (EVT/WOTE 13) (2013)
5. Brassard, G., Chaum, D., Crépeau, C.: Minimum disclosure proofs of knowledge. J. Comput. Syst. Sci. **37**(2), 156–189 (1988)
6. Bünz, B., Fisch, B., Szepieniec, A.: Transparent snarks from dark compilers. In: CRYPTO (2020)
7. Carback, R.T., Chaum, D., Clark, J., Conway, J., Essex, A., Hernson, P.S., Mayberry, T., Popoveniuc, S., Rivest, R.L., Shen, E., Sherman, A.T., Vora, P.L.: Scantegrity II municipal election at Takoma Park: the first E2E binding governmental election with ballot privacy. In: USENIX Security Symposium (2010)
8. Chaum, D., Carback, R., Clark, J., Essex, A., Popoveniuc, S., Rivest, R.L., Ryan, P.Y.A., Shen, E., Sherman, A.T.: Scantegrity II: end-to-end verifiability for optical scan election systems using invisible ink confirmation codes. In: EVT (2008)
9. Chaum, D., Carback, R., Clark, J., Essex, A., Popoveniuc, S., Rivest, R.L., Ryan, P.Y.A., Shen, E., Sherman, A.T., Vora, P.L.: Scantegrity ii: end-to-end verifiability by voters of optical scan elections through confirmation codes. IEEE Transactions on Information Forensics and Security **4**(4), 611–627 (2009)
10. Chiesa, A., Hu, Y., Maller, M., Mishra, P., Vesely, N., Ward, N.: Marlin: Preprocessing zksnarks with universal and updatable srs. In: CRYPTO (2020)
11. Culnane, C., Ryan, P.Y., Schneider, S., Teague, V.: vvote: a verifiable voting system. ACM Transactions on Information and System Security (TISSEC) **18**(1), 1–30 (2015)
12. Essex, A., Clark, J., Hengartner, U., Adams, C.: Eperio: Mitigating technical complexity in cryptographic election verification. In: EVT/WOTE (2010)
13. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive (2019)
14. Goldreich, O., Micali, S., Wigderson, A.: How to prove all np statements in zero-knowledge and a methodology of cryptographic protocol design. In: CRYPTO (1986)
15. Hao, F., Ryan, P.Y.: Real-world electronic voting: Design, analysis and deployment. CRC Press (2016)
16. Kusters, R., Truderung, T., Vogt, A.: Clash attacks on the verifiability of e-voting systems. In: IEEE Symposium on Security and Privacy (2012). https://doi.org/10.1109/SP.2012.32